Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O objetivo da atividade foi implementar um sistema de sincronização entre passageiros e vagões de metrô utilizando C e a biblioteca Pthreads. A implementação busca controlar a disponibilidade de assentos, permitir embarques concorrentes e garantir que os vagões só partam após a conclusão dos embarques autorizados. Para isso, foram utilizados mutexes e variáveis de condição.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Usar mutex para proteger o estado compartilhado da estação.	OK	
Usar variáveis de condição para bloquear threads sem busy-wait.	OK	
Permitir no máximo um carro na estação.	OK	
Passageiros aguardam até existir carro e assento livre.	OK	
Autorizar vários passageiros antes de qualquer confirmação on_board.	OK	
Carro aguarda as confirmações dos passageiros autorizados.	OK	
Carro parte quando está cheio ou não há mais passageiros esperando, após resolver autorizações pendentes.	OK	
Não utilizar busy-wait.	OK	

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar testes:

```
./sync_car
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. src/sync_car.c → Implementa a sincronização entre passageiros e vagões, controlando vagas disponíveis, embarques pendentes e condições de partida.
2. src/sync_car.h → Define a estrutura station, seus atributos e as declarações das funções de sincronização.
3. tests/sync_car-runner.c → Executa os testes oficiais, simulando múltiplos passageiros e vagões concorrentes.
4. Makefile → Automatiza a compilação do projeto e a geração do executável sync_car.
5. evidencias.log → Armazena os registros de compilação, execução e resultados dos testes.

Decisões técnicas: Decisão → Motivo → Consequência

1. Utilização de um mutex por estação → Proteger o acesso às variáveis compartilhadas → Evita condições de corrida e mantém a consistência dos dados.
2. Utilização de variáveis de condição (pthread_cond_t) → Permitir que passageiros e vagões aguardem eventos sem espera ativa → Reduz o consumo desnecessário de CPU e permite a sincronização eficiente.
3. Controle de assentos disponíveis e embarques pendentes → Diferenciar a reserva de uma vaga da conclusão efetiva do embarque → Impede que o vagão parta antes de todos os passageiros autorizados concluírem o embarque.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Estudar conceitos de sincronização	Percebi que antes de implementar o código precisava adquirir o conhecimento lógico e técnico.	
S2	Construção incremental	Implementar por partes, visando melhor entendimento e menos erros.	

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Estudar sobre sincronização	OK		30/10	
2	S1/S2	Definir a struct station e seus parâmetros	OK		01/11	
3	S2	Implementar station_init	OK		01/11	evidencias.log 27-109
4	S2	Sinalizar cond_vagao após atualizar o estado compartilhado	OK		01/11	evidencias.log 110-145
5.1	S2	Primeira implementação de station_load_car	PARCIAL		03/11	evidencias.log 156-182
5.2	S2	Correção dos erros de compilação em station_load_car	OK		03/11	evidencias.log 209-218
6	S2	Implementar station_load_car e sua condição de partida	OK		03/11	evidencias.log 245-310

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

Durante a implementação da função station_load_car, encontrei erros de compilação relacionados ao uso incorreto de pthread_cond_broadcast e ao parâmetro count, que não estava sendo utilizado corretamente. Após analisar as mensagens do compilador, corrigi a chamada da variável de condição e ajustei a utilização do parâmetro para definir a quantidade de assentos disponíveis. Em seguida, compilei novamente o código e obtive código de saída 0, confirmando que os erros de compilação foram resolvidos.

Erro:

```
luizhlcosta@DESKTOP-233HMBM:/mnt/c/Users/luizh/Sincronizacao_do_Metro_SO$ gcc -Wall -Wextra -Werror -pthread -c src/sync_car.c -o sync_car.o

echo "Codigo de saida: $?"
src/sync_car.c: In function ‘station_load_car’:
src/sync_car.c:59:28: error: passing argument 1 of ‘pthread_cond_broadcast’ from incompatible pointer type [-Werror=incompatible-pointer-types]
   59 |     pthread_cond_broadcast(&station->passageiros_aguardando);
      |                            ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
      |                            |
      |                            int *
In file included from src/sync_car.c:3:
/usr/include/pthread.h:1125:52: note: expected ‘pthread_cond_t *’ but argument is of type ‘int *’
 1125 | extern int pthread_cond_broadcast (pthread_cond_t *__cond)
      |                                    ~~~~~~~~~~~~~~~~^~~~~~
src/sync_car.c:54:45: error: parameter ‘count’ set but not used [-Werror=unused-but-set-parameter]
   54 | void station_load_car(station *station, int count) {
      |                                         ~~~~^~~~~
cc1: all warnings being treated as errors
Codigo de saida: 1
luizhlcosta@DESKTOP-233HMBM:/mnt/c/Users/luizh/Sincronizacao_do_Metro_SO$
```

Após solução:

```
luizhlcosta@DESKTOP-233HMBM:/mnt/c/Users/luizh/Sincronizacao_do_Metro_SO$ gcc -Wall -Wextra -Werror -pthread -c src/sync_car.c -o sync_car.o

echo "Codigo de saida: $?"
Codigo de saida: 0
luizhlcosta@DESKTOP-233HMBM:/mnt/c/Users/luizh/Sincronizacao_do_Metro_SO$
```

7. Uso de IA

Onde usei IA (2–3 linhas):
Utilizei a IA como ferramenta de apoio para compreender os conceitos de sincronização, discutir alternativas de implementação e identificar possíveis erros. As decisões foram desenvolvidas e revisadas ao longo da atividade, com validação por meio de compilações e testes executados no ambiente de desenvolvimento.

Prompts principais (liste, sem história):

- “Como funcionam as funções da biblioteca pthreads apresentadas na atividade.”
- “A struct station que eu criei faz sentido no contexto dessa atividade? Valide minha tentativa sem entregar a resposta correta.”

O que validei manualmente e como:
Realizei os testes fornecidos pelo professor e o funcionamento de algumas funções específicas, para entender o momento correto de chamá-las no meu código.

8. Reflexão Final (5–8 linhas)

Reflexão:

Durante a atividade, consegui compreender de forma mais clara o funcionamento das funções apresentadas (pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, pthread_cond_init, pthread_cond_signal, pthread_cond_wait e pthread_cond_broadcast) e as sutis diferenças que elas apresentam, algo que possibilitou consolidar o conhecimento de forma mais assertiva.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	SIM
Rodei os testes e evidencias.log foi gerado	SIM
Tenho prints obrigatórios	SIM
Testei pelo menos um caso limite e um caso inválido	SIM
Preenchi a seção de uso de IA	SIM
Revisei o relatório e removi frases genéricas / vazias	SIM

10. Se eu tivesse mais 2 horas

Com um pouco mais de tempo, meu próximo passo seria tentar criar mais testes, além daqueles enviados pelo professor.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.